

# Lab Assignment

## International Cybersecurity and Digital Forensics Academy

"Preparing Global Cybersecurity Leaders Through Excellence in Hands-On Training"

### 1. Purpose of the Lab Assignment

The purpose of this lab is to understand how a Linux host-based firewall controls network traffic. It focuses on using iptables to allow or block HTTP traffic from a specific client. The lab also requires capturing allowed and blocked traffic using tshark or Wireshark. The captured traffic is examined to identify differences between successful and blocked connections. The exercise also demonstrates how firewall rules and packet evidence can support digital forensic investigations.

### 2. Lab Report Structure

The report covers the lab network setup, original firewall rules and evidence hashes. It documents the allowed HTTP baseline and the implementation of the blocking rule. The report then explains the blocked traffic and compares it with the allowed capture. It also discusses DROP versus REJECT and the importance of firewall counters.

#### 2.1. Title Page

- Lab Title: LAB 6: Firewall Traffic Control and Forensic Verification
- Student Name: Olanrewaju Wale Mustapha Oladimeji.
- Student ID: 2025/FWSD/11320.
- Course Name: SBT-DF203: Basic Networking Skills for Digital Forensics
- Instructor Name: Aminu Idris
- Date of Submission: 19/09/2026
- Version:

#### 2.2. Executive Summary

This lab investigated how a Linux firewall controls HTTP traffic from different clients. A baseline HTTP connection would be captured before applying the firewall rule.

## 2.3. Lab Objectives

The objectives are to understand host-based and network-based firewalls and interpret iptables chains. The lab requires creating, checking and removing a narrowly scoped firewall rule. It also requires capturing both allowed and blocked HTTP connection attempts.

## 2.4. Tools and Resources Used

Kali Linux : was used as the main forensic working environment.

Kali Linux was used as the main operating system for performing the practical work.

Apache was used as the local web service on 127.0.0.1:80.

tshark and Wireshark were used to capture and examine the network packets.

Python3 with Scapy was used to generate the small number of authorized SYN packets.

SHA-256 hashing was used to help confirm that the original evidence was preserved.

## 2.5. Methodology

### 6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
```

```
cd ~/SBT-DF203-Lab6
```

```
pwd
```

```
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
```

```
sudo apt install -y apache2 curl iptables tshark wireshark
```

```
sudo systemctl enable --now apache2
```

```
printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall
```

```
Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>\n' | sudo tee
```

```
/var/www/html/firewall_lab.html
```

```
sudo iptables-save | tee reports/iptables_before.rules
```

```
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
```

```
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
```

```
water@kali:~$ mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
water@kali:~$ cd ~/SBT-DF203-Lab6
water@kali:~/SBT-DF203-Lab6$ pwd
/home/water/SBT-DF203-Lab6
water@kali:~/SBT-DF203-Lab6$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./reports
./screenshots
./scripts
water@kali:~/SBT-DF203-Lab6$
```

```
water@kali:~/SBT-DF203-Lab6$ sudo systemctl enable --now apache2
[sudo] password for water:
water@kali:~/SBT-DF203-Lab6$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-09-19 14:48:33 EDT; 19h ago
     Invocation: 7c7cc820abdd45c8a892b5b322d0ba7
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 1010 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 7 (limit: 4403)
       Memory: 23.4M (peak: 37.5M)
         CPU: 1.974s
    CGroup: /system.slice/apache2.service
            └─ 1010 /usr/sbin/apache2 -k start -DFOREGROUND
               18875 /usr/sbin/apache2 -k start -DFOREGROUND
               18876 /usr/sbin/apache2 -k start -DFOREGROUND
               18877 /usr/sbin/apache2 -k start -DFOREGROUND
               18878 /usr/sbin/apache2 -k start -DFOREGROUND
               18879 /usr/sbin/apache2 -k start -DFOREGROUND
               18880 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 19 14:48:33 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 19 14:48:33 kali apache2ctl[1010]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress thi
Sep 19 14:48:33 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Sep 19 15:35:48 kali systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Sep 19 15:35:48 kali apache2ctl[10986]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
Sep 19 15:35:48 kali systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
Sep 20 09:09:19 kali systemd[1]: Reloading apache2.service - The Apache HTTP Server...
Sep 20 09:09:19 kali apache2ctl[48841]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
Sep 20 09:09:19 kali systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
lines 1-23/23 (END)
```

```
water@kali:~/SBT-DF203-Lab6$ printf '<DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
<DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>
water@kali:~/SBT-DF203-Lab6$ curl http://127.0.0.1/firewall_lab.html
<DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>
water@kali:~/SBT-DF203-Lab6$
```

```
water@kali:~/SBT-DF203-Lab6$ sudo iptables-save | tee reports/iptables_before.rules
# Generated by iptables-save v1.8.13 (nf_tables) on Sun Sep 20 10:30:37 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
COMMIT
# Completed on Sun Sep 20 10:30:37 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Sun Sep 20 10:30:37 2026
*nat
:PREROUTING ACCEPT [40:5060]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [119:22056]
:POSTROUTING ACCEPT [114:20786]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Sun Sep 20 10:30:37 2026
water@kali:~/SBT-DF203-Lab6$
```

```
(water@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination
Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1    0    0 DOCKER-USER all  --  *      *        0.0.0.0/0         0.0.0.0/0
2    0    0 DOCKER-FORWARD all --  *      *        0.0.0.0/0         0.0.0.0/0
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination
Chain DOCKER (1 references)
num  pkts bytes target    prot opt in     out     source            destination
1    0    0 DROP     all  --  !docker0 docker0 0.0.0.0/0         0.0.0.0/0
Chain DOCKER-BRIDGE (1 references)
num  pkts bytes target    prot opt in     out     source            destination
1    0    0 DOCKER   all  --  *      *        0.0.0.0/0         0.0.0.0/0
Chain DOCKER-CT (1 references)
num  pkts bytes target    prot opt in     out     source            destination
1    0    0 ACCEPT   all  --  *      *        0.0.0.0/0         0.0.0.0/0
Chain DOCKER-FORWARD (1 references)
num  pkts bytes target    prot opt in     out     source            destination
1    0    0 DOCKER-CT all  --  *      *        0.0.0.0/0         0.0.0.0/0
2    0    0 DOCKER-ISOLATION-STAGE-1 all --  *      *        0.0.0.0/0         0.0.0.0/0
3    0    0 DOCKER-BRIDGE all --  *      *        0.0.0.0/0         0.0.0.0/0
4    0    0 ACCEPT   all  --  docker0 *        0.0.0.0/0         0.0.0.0/0
Chain DOCKER-ISOLATION-STAGE-1 (1 references)
num  pkts bytes target    prot opt in     out     source            destination
1    0    0 DOCKER-ISOLATION-STAGE-2 all --  docker0 !docker0 0.0.0.0/0         0.0.0.0/0
Chain DOCKER-ISOLATION-STAGE-2 (1 references)
num  pkts bytes target    prot opt in     out     source            destination
1    0    0 DROP     all  --  *      *        0.0.0.0/0         0.0.0.0/0
Chain DOCKER-USER (1 references)
num  pkts bytes target    prot opt in     out     source            destination
ctstate RELATED,ESTABLISHED

(water@kali)-[~/SBT-DF203-Lab6]
$
```

```
(water@kali)-[~/SBT-DF203-Lab6]
$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
62163cd35660910eda5f518d82f032d12ab8685a704825f62d1f6372259ab24a  reports/iptables_before.rules

(water@kali)-[~/SBT-DF203-Lab6]
$
```

```
(water@kali)-[~/SBT-DF203-Lab6]
$ ls -lh reports/
total 12K
-rw-rw-r-- 1 water water 1.3K Sep 20 10:30 iptables_before.rules
-rw-rw-r-- 1 water water  96 Sep 20 10:33 iptables_before_sha256.txt
-rw-rw-r-- 1 water water 2.5K Sep 20 10:32 iptables_before.txt

(water@kali)-[~/SBT-DF203-Lab6]
$ find ~/SBT-DF203-Lab6 -maxdepth 2 -type f -print
/home/water/SBT-DF203-Lab6/reports/iptables_before.txt
/home/water/SBT-DF203-Lab6/reports/iptables_before.rules
/home/water/SBT-DF203-Lab6/reports/iptables_before_sha256.txt

(water@kali)-[~/SBT-DF203-Lab6]
$
```

7. Mini Evidence and Chain-of-Custody Worksheet

| Field                 | Student Entry                                                      |
|-----------------------|--------------------------------------------------------------------|
| Case/lab identifier   | SBT-DF203-Lab6- Firewall Traffic Control and Forensic Verification |
| Trainee name          | Olanrewaju Wale Mustapha Oladimeji                                 |
| Date and time started | 20 September 2026 3:35pm                                           |

| Field                       | Student Entry                                                                                                                                                                                                                            |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Evidence file name(s)       | reports/iptables_before.rules<br>reports/iptables_before.txt<br>reports/iptables_before_sha256.txt                                                                                                                                       |
| Source or generation method | Original firewall rules were exported using sudo iptables-save; the rules listing was generated using sudo iptables -L -n -v --line-numbers.                                                                                             |
| Original SHA-256            | 62163cd35660910eda5f518d82f032d12ab8685a704825f62d1f6372259ab24a reports/iptables_before.rules                                                                                                                                           |
| Working-copy SHA-256        |                                                                                                                                                                                                                                          |
| Analysis workstation/VM     | Kali Linux VM — /home/water/SBT-DF203-Lab6                                                                                                                                                                                               |
| Notes on any changes        | Lab folders were created for evidence preservation and analysis. Apache was installed and started, the firewall ruleset was exported before changes, and the SHA-256 hash was generated. No blocking firewall rule has been applied yet. |

## 8. Part A - Record the Lab Network and Baseline Access

# On the server

ip -br address | tee reports/server\_interfaces.txt

ip route | tee reports/server\_routes.txt

sudo ss -lntp | grep ':80' | tee reports/apache\_listener.txt

# On the client, replace SERVER\_IP

curl -v --connect-timeout 5 http://SERVER\_IP/firewall\_lab.html 2>&1 | tee baseline\_curl.txt

Record the server IP, allowed client IP, blocked client IP, interface and default gateway. Take a snapshot before changing the firewall.

```
(water@kali)~/SBT-DF203-Lab6
$ ip -br address | tee reports/server_interfaces.txt
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        UP          192.168.244.128/24 fe80::20c:29ff:fe53:93b5/64
docker0     DOWN       172.17.0.1/16

(water@kali)~/SBT-DF203-Lab6
$ cat reports/server_interfaces.txt
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        UP          192.168.244.128/24 fe80::20c:29ff:fe53:93b5/64
docker0     DOWN       172.17.0.1/16

(water@kali)~/SBT-DF203-Lab6
$ ip route | tee reports/server_routes.txt
default via 192.168.244.2 dev eth0 proto dhcp src 192.168.244.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.244.0/24 dev eth0 proto kernel scope link src 192.168.244.128 metric 100

(water@kali)~/SBT-DF203-Lab6
$ cat reports/server_routes.txt
default via 192.168.244.2 dev eth0 proto dhcp src 192.168.244.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.244.0/24 dev eth0 proto kernel scope link src 192.168.244.128 metric 100

(water@kali)~/SBT-DF203-Lab6
$
```

```
(water@kali)~/SBT-DF203-Lab6
$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0      511          *:80          users:((("apache2",pid=48880,fd=4),("apache2",pid=48879,fd=4),("apache2",pid=48878,fd=4),("apache2",pid=48877,fd=4),("apache2",pid=48876,fd=4),("apache2",pid=48875,fd=4),("apache2",pid=1010,fd=4)))

(water@kali)~/SBT-DF203-Lab6
$ ip -br address
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        UP          192.168.244.128/24 fe80::20c:29ff:fe53:93b5/64
docker0     DOWN       172.17.0.1/16

(water@kali)~/SBT-DF203-Lab6
$ curl -v --connect-timeout 5 http://192.168.244.128/firewall_lab.html 2>&1 | tee baseline_curl.txt
* Trying 192.168.244.128:80...
* Established connection to 192.168.244.128 (192.168.244.128 port 80) from 192.168.244.128 port 45450
* Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left   Speed
0    0    0    0    0    0     0      0      0  0  0  0  0
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.244.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sun, 20 Sep 2026 14:47:31 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sun, 20 Sep 2026 14:28:25 GMT
< ETag: "7f-05beaf24ca320"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [119 bytes data]
100 119 100 119 0 0 65348 0 0
* Connection #0 to host 192.168.244.128:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>

(water@kali)~/SBT-DF203-Lab6
$
```

## 9. Part B - Capture the Allowed HTTP Baseline

# On server, replace IFACE and CLIENT\_IP

sudo tshark -i IFACE -f 'host CLIENT\_IP and tcp port 80' -a duration:30 -w

evidence/http\_allowed.pcapng &

sleep 3

# Run curl from client during this capture

wait

sha256sum evidence/http\_allowed.pcapng | tee reports/http\_allowed\_sha256.txt

```
(water@kali)-[~/SBT-DF203-Lab6]
$ sudo tshark -i eth0 -f 'host 192.168.244.128 and tcp port 80' -a duration:30 -w evidence/http_allowed.pcapng 6
[1] 68318

(water@kali)-[~/SBT-DF203-Lab6]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

[1] + done      sudo tshark -i eth0 -f 'host 192.168.244.128 and tcp port 80' -a duration:30
(water@kali)-[~/SBT-DF203-Lab6]
$

(water@kali)-[~/SBT-DF203-Lab6]
$ ls -l evidence/http_allowed.pcapng
-rw-r----- 1 water water 372 Sep 20 12:27 evidence/http_allowed.pcapng

(water@kali)-[~/SBT-DF203-Lab6]
$ sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
26ef9011643196604fdcbd59a8d3356664800fcf4627687c69d6fc0c44e66c34  evidence/http_allowed.pcapng

(water@kali)-[~/SBT-DF203-Lab6]
$
```

## 10. Part C - Apply and Verify the Blocking Rule

Use the exact source IP of the assigned blocked client. Insert the rule near the top of INPUT so it is evaluated before a broad ACCEPT rule.

BLOCKED\_CLIENT\_IP=192.168.56.20

```
sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
```

```
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
```

```
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
```

```
printf 'Rule verified present.\n' | tee reports/rule_verification.txt
```

```
(water@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.244.128

(water@kali)-[~/SBT-DF203-Lab6]
$ echo "$BLOCKED_CLIENT_IP"
192.168.244.128

(water@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

(water@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
Chain INPUT (policy ACCEPT 1 packets, 76 bytes)
num  pkts bytes target    prot opt in     out     source                destination            tcp dpt:80
1      0      0 DROP      tcp  --  *      *      192.168.244.128        0.0.0.0/0

(water@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

(water@kali)-[~/SBT-DF203-Lab6]
$ printf 'Rule verified present.\n' | tee reports/rule_verification.txt
Rule verified present.

(water@kali)-[~/SBT-DF203-Lab6]
$
```

## 11. Part D - Capture Blocked Traffic and Rule Counters

# On server

```
sudo tshark -i IFACE -f 'host 192.168.56.20 and tcp port 80' -a duration:35 -w
```

```
evidence/http_blocked.pcapng &
```

```
sleep 3
```

# From blocked client during capture:

# curl -v --connect-timeout 10 http://SERVER\_IP/firewall\_lab.html

wait

sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables\_after\_test.txt

sha256sum evidence/http\_blocked.pcapng | tee reports/http\_blocked\_sha256.txt

```
(water@kali)~[~/SBT-DF203-Lab6]
$ ip -br addr
lo          UNKNOWN    127.0.0.1/8 ::1/128
eth0        UP          192.168.244.128/24 fe80::20c:29ff:fe53:93b5/64
docker0     DOWN       172.17.0.1/16

(water@kali)~[~/SBT-DF203-Lab6]
$ ip -4 addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   inet 192.168.244.128/24 brd 192.168.244.255 scope global dynamic noprefixroute eth0
       valid_lft 1407sec preferred_lft 1407sec
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever

(water@kali)~[~/SBT-DF203-Lab6]
$ sudo tshark -i eth0 -r 'host 192.168.244.128 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng
[1] 74396

(water@kali)~[~/SBT-DF203-Lab6]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

[1] + done      sudo tshark -i eth0 -f 'host 192.168.244.128 and tcp port 80' -a duration:35
(water@kali)~[~/SBT-DF203-Lab6]
$ sleep 3

(water@kali)~[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 10 http://SERVER_IP/firewall_lab.html
* Resolving timed out after 10002 milliseconds
* closing connection #0
curl: (28) Resolving timed out after 10002 milliseconds
```

```
(water@kali)~[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 10 http://192.168.244.128/firewall_lab.html
* Trying 192.168.244.128:80...
* Established connection to 192.168.244.128 (192.168.244.128 port 80) from 192.168.244.128 port 59006
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.244.128
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sun, 20 Sep 2026 17:16:11 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Sun, 20 Sep 2026 14:28:25 GMT
< ETag: "77-65beaf24ca320"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><div>ICDFA Network Forensics Firewall Lab</div><p>Server: YOUR SERVER NAME</p></body></html>
* Connection #0 to host 192.168.244.128:80 left intact

(water@kali)~[~/SBT-DF203-Lab6]
$
```

```
(water@kali)~[~/SBT-DF203-Lab6]
$ ls -lh evidence/http_blocked.pcapng
-rw-r----- 1 root root 372 Sep 20 12:44 evidence/http_blocked.pcapng

(water@kali)~[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
Chain INPUT (policy ACCEPT 20 packets, 1528 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 DROP      tcp  --  *      *        192.168.244.128      0.0.0.0/0           tcp dpt:80

(water@kali)~[~/SBT-DF203-Lab6]
```

```
(water@kali)~[~/SBT-DF203-Lab6]
$ ls -l evidence/http_blocked.pcapng
-rw-r----- 1 root root 372 Sep 20 12:44 evidence/http_blocked.pcapng

(water@kali)~[~/SBT-DF203-Lab6]
$ sudo chown water:water evidence/http_blocked.pcapng

(water@kali)~[~/SBT-DF203-Lab6]
$ ls -l evidence/http_blocked.pcapng
-rw-r----- 1 water water 372 Sep 20 12:44 evidence/http_blocked.pcapng

(water@kali)~[~/SBT-DF203-Lab6]
$ sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
b7a9476122c7c1949c8c147bf3070920311443b2478b87f3140d6f371539f7  evidence/http_blocked.pcapng

(water@kali)~[~/SBT-DF203-Lab6]
$
```

## 12. Part E - Compare Allowed and Blocked Captures

for PCAP in evidence/http\_allowed.pcapng evidence/http\_blocked.pcapng; do

```
echo "==== $PCAP ===="
```

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response ||
```

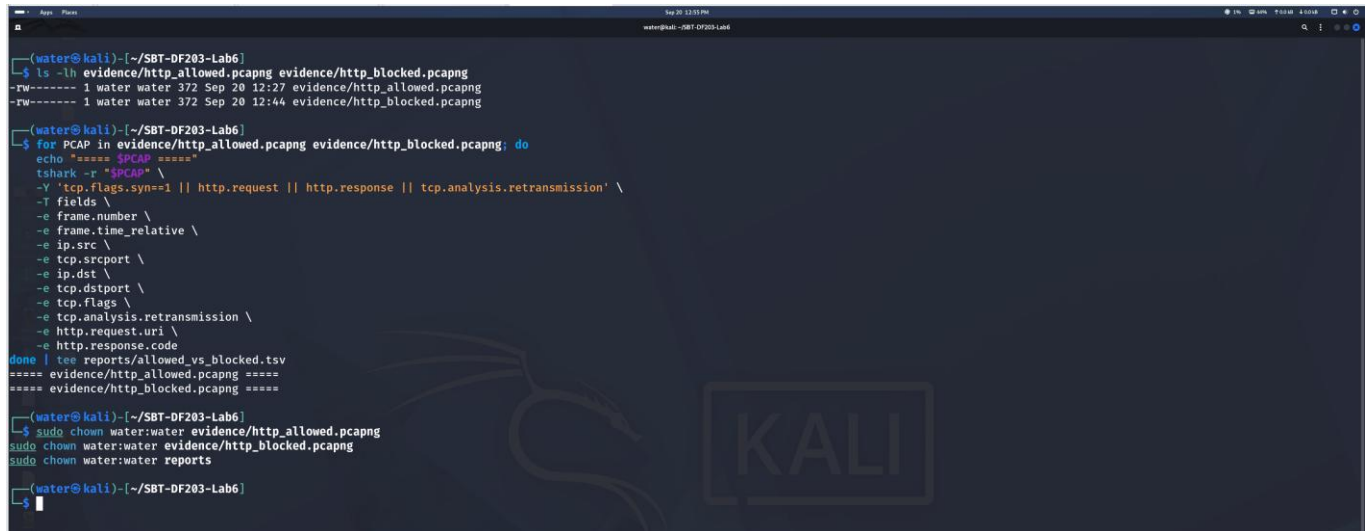
```
tcp.analysis.retransmission' \
```

```
-T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport
```

```
\
```

```
-e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
```

```
done | tee reports/allowed_vs_blocked.tsv
```



```
(water@kali)~/SBT-DF203-Lab6
$ ls -lh evidence/http_allowed.pcapng evidence/http_blocked.pcapng
-rw-r--r-- 1 water water 372 Sep 20 12:27 evidence/http_allowed.pcapng
-rw-r--r-- 1 water water 372 Sep 20 12:44 evidence/http_blocked.pcapng

(water@kali)~/SBT-DF203-Lab6
$ for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
echo "==== $PCAP ===="
tshark -r "$PCAP" \
-Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags \
-e tcp.analysis.retransmission \
-e http.request.uri \
-e http.response.code
done | tee reports/allowed_vs_blocked.tsv
==== evidence/http_allowed.pcapng ====
==== evidence/http_blocked.pcapng ====

(water@kali)~/SBT-DF203-Lab6
$ sudo chown water:water evidence/http_allowed.pcapng
sudo chown water:water evidence/http_blocked.pcapng
sudo chown water:water reports
(water@kali)~/SBT-DF203-Lab6
```

## 13. Part F - Remove the Rule and Restore Access

BLOCKED\_CLIENT\_IP=192.168.56.20

```
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
```

```
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule
successfully removed'
```

```
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
```

```
# Confirm access from the previously blocked client
```

```
# curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html
```

```

(water@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.244.128

(water@kali)-[~/SBT-DF203-Lab6]
$ echo "$BLOCKED_CLIENT_IP"
192.168.244.128

(water@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

(water@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed

(water@kali)-[~/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
Chain INPUT (policy ACCEPT 25 packets, 2360 bytes)
num pkts bytes target prot opt in out source destination

(water@kali)-[~/SBT-DF203-Lab6]
$ cat reports/iptables_restored.txt
Chain INPUT (policy ACCEPT 25 packets, 2360 bytes)
num pkts bytes target prot opt in out source destination

(water@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 http://192.168.244.128/firewall_lab.html 2>&1 | tee baseline_curl.txt
* Trying 192.168.244.128:80...
* Established connection to 192.168.244.128 (192.168.244.128 port 80) from 192.168.244.128 port 38394
% Total % Received % Xferd Average Speed Time Time Time Current
Load Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 * using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.244.128
> User-Agent: curl/8.21.0
> Accept: */*
> 
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sun, 20 Sep 2026 17:12:26 GMT
< Server: Apache/2.4.66 (Debian)
< Last-Modified: Sun, 20 Sep 2026 14:28:25 GMT
< ETag: "77-65beaf24ca320"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
< 
[119 bytes data]
100 119 100 119 0 0 83921 0 0
* Connection #0 to host 192.168.244.128:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>

(water@kali)-[~/SBT-DF203-Lab6]
$

```

## 15. Forensic Interpretation Questions

1. Why does DROP commonly cause SYN retransmissions and a timeout?

A DROP rule silently discards the client's connection request. The client does not receive a reply from the server; it therefore sends the SYN packet again after waiting. This repeated attempt means SYN retransmission. After several attempts, the client eventually reports a connection timeout.

2. How would a REJECT rule differ in the packet capture and client output?

A REJECT rule sends a response back to the client instead of silently dropping the packet, then the client therefore receives an immediate response that the connection is not allowed. The packet capture can show the rejection response from the server or firewall, So, the client may report “Connection refused” or a similar error. Unlike DROP, REJECT normally does not require the client to wait for a long timeout.

- ### 3. Why are firewall counters valuable corroborating evidence?

Firewall counters show how many packets and bytes matched a particular rule. An increasing counter supports the finding that traffic actually reached and matched the rule.

This provides evidence in addition to what was seen in the PCAP. The counters can help connect the firewall activity with the client's test and they are useful for confirming that the blocking rule was actively used.

4. What evidence would show that the web server was down rather than firewall-blocked?

The server could show that the web service was not running on TCP port 80, a test from an allowed client could also fail to connect to the same server. The firewall rule counters would remain unchanged if no traffic matched the rule then server logs could show that no HTTP request was received.

5. What is the risk of deleting a rule by line number after other rules have changed?

Firewall rules are processed in a specific numbered order, if rules are added or removed, their line numbers can change. Deleting by an old line number could therefore remove the wrong rule and this might unintentionally allow or block traffic.

Deleting by matching the rule's exact conditions is generally safer.

## **2.7. Analysis and Findings**

The lab showed how an iptables firewall rule can be used to block HTTP traffic from a specific client IP address. The DROP rule was added to the INPUT chain for client 192.168.244.128 and TCP port 80. The firewall rule was successfully verified, and the rule was later removed to restore access.

## **2.8. Challenges and Solutions**

One challenge was getting permission errors when reading the captured PCAP files. This happened because files created with sudo tshark were initially owned by root. The problem was solved by changing the ownership of the PCAP files to the water user.

## **2.9. Conclusion**

The lab demonstrated the basic process of using iptables to control HTTP traffic from a selected client. The DROP rule was inserted near the top of the INPUT chain and was successfully verified. The rule was later deleted, and the firewall configuration was checked after restoration.

## **2.10. Recommendations**

Assignment like this improved student knowledge and understanding of Network trafficking

## **2.11. References**

Class Video Guide from Founder ICDFA on Basic Networking Skills for Digital Forensics

Network protocol analyzer guide from <https://www.wireshark.org>